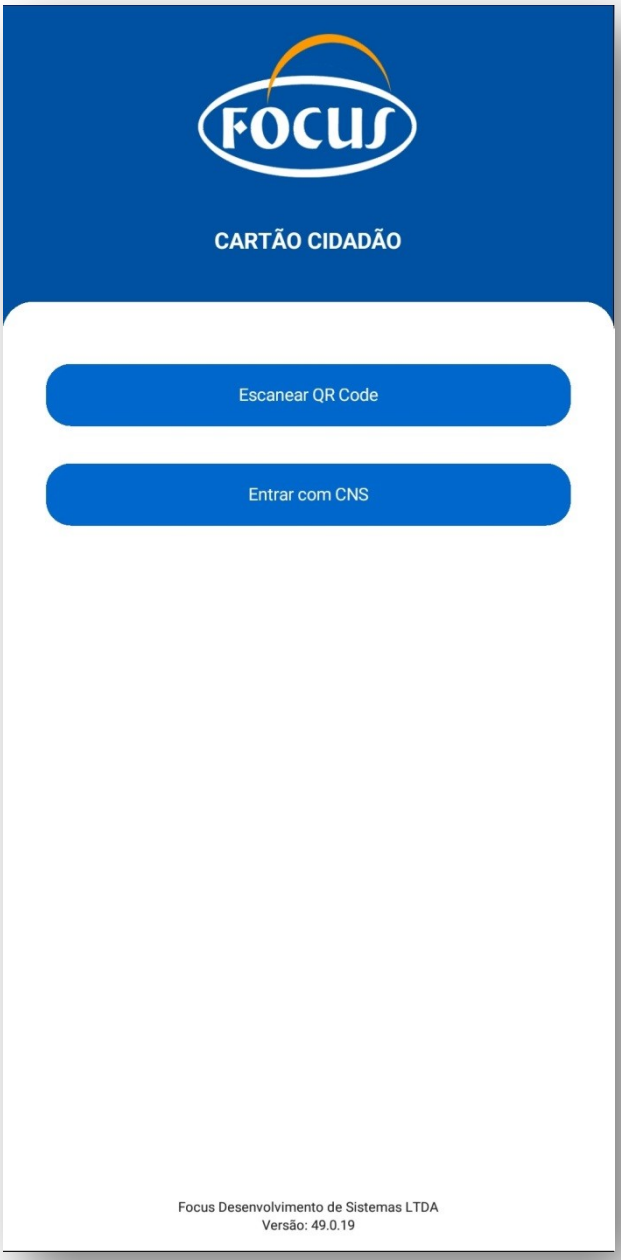
Manual de Login — App Delphi FMX (fluxos)

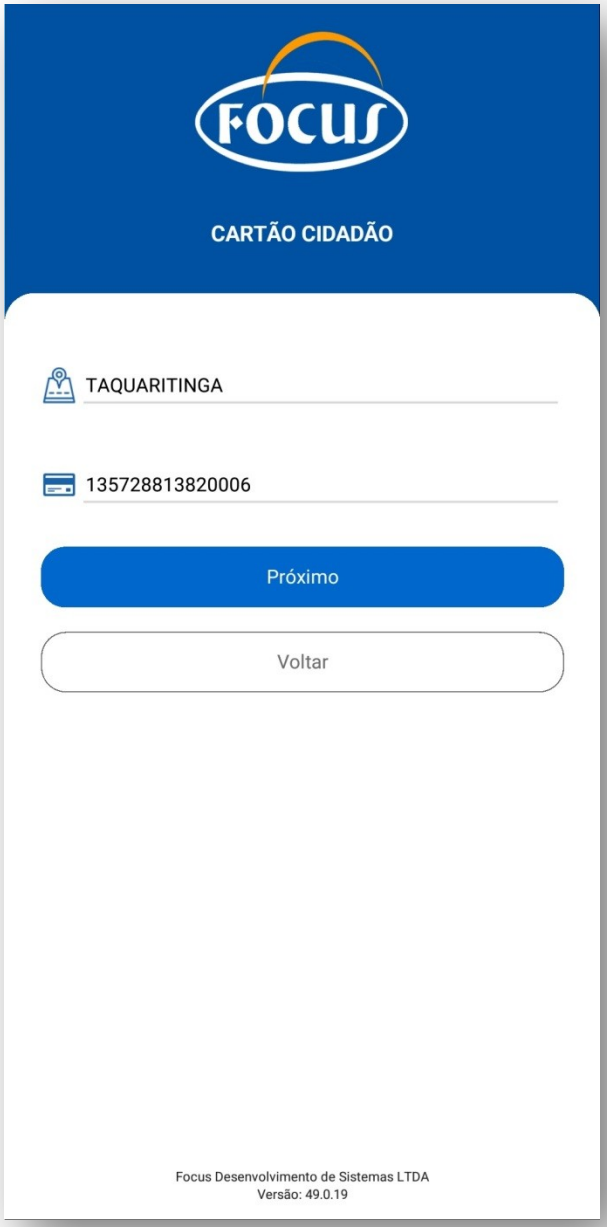
Visão geral dos fluxos de login:

1. **O app faz uma requisição GET para a rota para trazer a prefeitura:**
   GET - http://[UrlPadrao]/cartao-cidadao/focus/prefeitura/municipio



- Recebe um array JSON com os municípios (codPREF, nomeMUN, codIBGE, descricao).
- Para cada município retornado:
- Verifica no banco local (TAB_MUNICIPIO) se já existe o registro com aquele CODPREF.
- Se **não existir**, faz um INSERT no banco local com os dados do município.
- Se **já existir**, ignora (não atualiza nem duplica).
- No final, o app garante que o banco local tem a lista completa de municípios válidos.

2. **O app faz uma requisição GET para a rota pra trazer o cidadão:**
GET - http://[Conexao.Url]/cartao-Cidadao/[Conexao.Banco]/cidadao/cns/[Usuario.CNS]



- Validação da resposta:
- Se o StatusCode for diferente de 200, gera erro:
  "Inválido, erro ao acessar cidadão!"

3. **Leitura e decodificação dos dados:**
   - Converte o conteúdo da resposta (Response.Content) em UTF-8.
   - Faz o *parse* do conteúdo para um TJSONArray.
   - Garante que o array não esteja vazio antes de prosseguir.

4. **Preenchimento do objeto Usuario:**
   - Lê o primeiro item do JSON (JsonObj := JsonAry.Items[0]).
   - Preenche todas as propriedades principais do cidadão:
     - Codigo, Nome, CNS, CPF, Sexo, Mae, DataNascimento
     - Senha, Telefone, Email, Token
     - TipoValidacao (define se será validação via senha, SMS, e-mail, ou WhatsApp)
   - Também armazena DTAtualizacao e Tokken (token de autenticação).

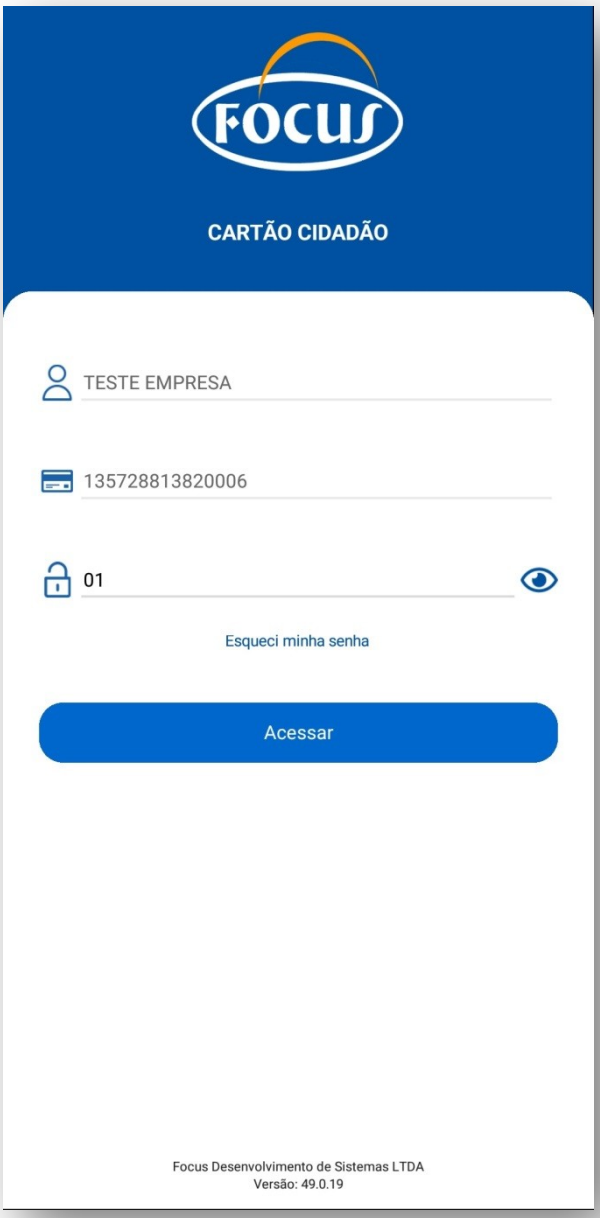
5. **Leitura do endereço (subobjeto):**
   - Se houver objeto endereco no JSON, preenche: CEP, Logradouro, Numero, Complemento, Bairro, Municipio
   - Tratamento de erro:
   - Qualquer falha no processamento gera uma exceção:
   "Erro ao processar os dados do cidadão: [mensagem]"

- Verifica se os campos CNS e Município foram preenchidos.
- Usa ValidarCNS para confirmar se o número do CNS é válido.
- Remove acentos e caracteres especiais do nome do município (RetiraCaracteresEspeciais).

6. **Consulta ao município local:**
   - Busca o município digitado na tabela local TAB_MUNICIPIO.
   - Se não encontrar, mostra mensagem de erro e interrompe o processo.
   - Obtém o CODPREF (código da prefeitura) correspondente.

7. **Busca da conexão e dados do cidadão:**

- Chama dmPrincipal.PesquisarConexao(Encrypt(CodPref.ToString))
  → configura a **base ou endpoint** da prefeitura selecionada.



- Define Usuario.Cns e chama dmCidadao.PesquisarCidadao
  → obtém os **dados cadastrais** do cidadão no servidor da prefeitura.

- Caso o cidadão já tenha senha cadastrada:

- Avança diretamente para a aba tabLogin.

- Preenche os campos edNomeLogin e edCNSLogin com os dados do cidadão.

8. **Validação da senha digitada:**

- Verifica se o campo edSenhaLogin está vazio.
- Compara a senha digitada com a senha armazenada em Usuario.Senha.
- Usa Decrypt(Usuario.Senha) para decifrar a senha salva.
- Usa Cripitografe(...) para comparar de forma segura (criptografada).
- Se estiver incorreta, exibe mensagem e interrompe o processo.

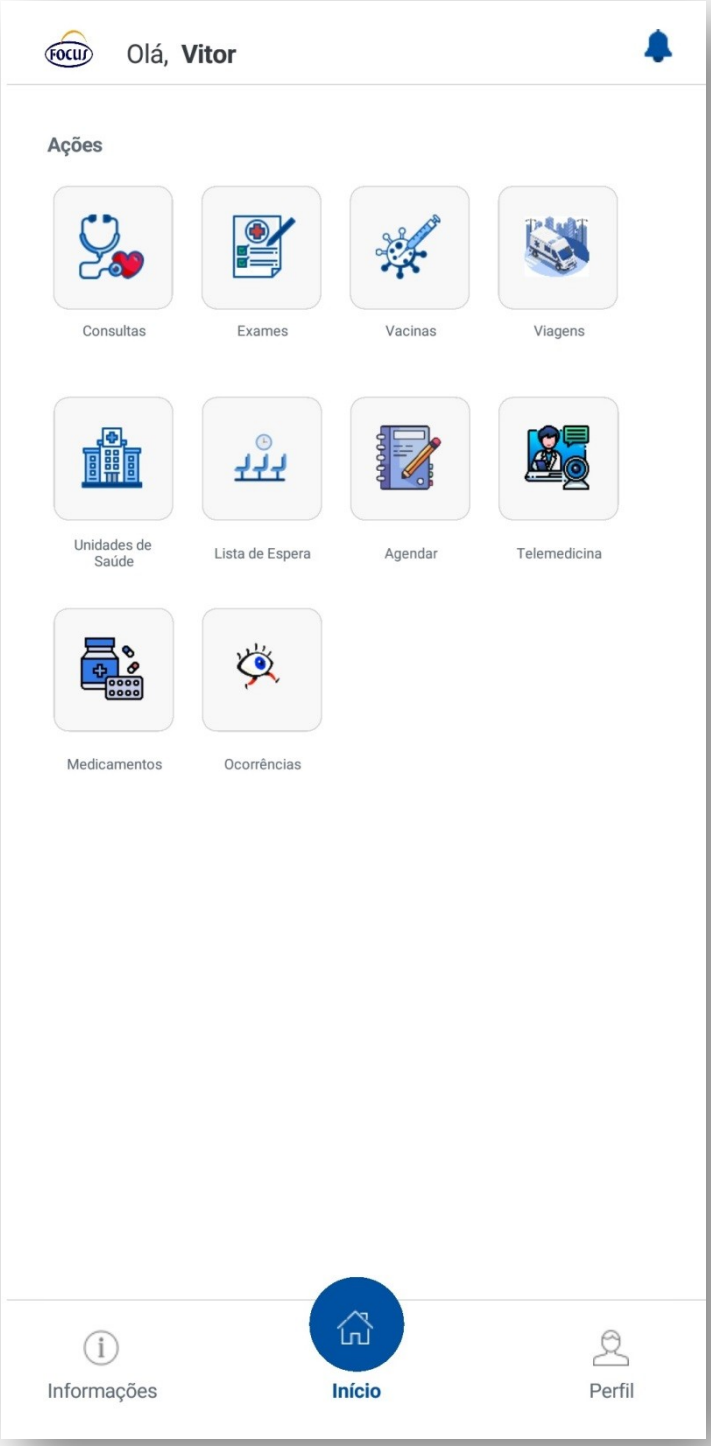
9. **Carga dos dados locais:**

- Após validação bem-sucedida, chama:

- dmPrincipal.PopulaBancoConexaoLocal
  → grava as informações de conexão da prefeitura ativa no banco local.

- dmCidadao.PopulaBancoCidadaoLocal
  → grava os dados do cidadão logado localmente.

- dmCidadao.PopulaBancoDependenteLocal
  → grava os dependentes do cidadão (caso existam).



- Essas tabelas locais permitem o funcionamento **offline** do app e mantêm o contexto do login.

- **Inicialização da interface**

  principal:

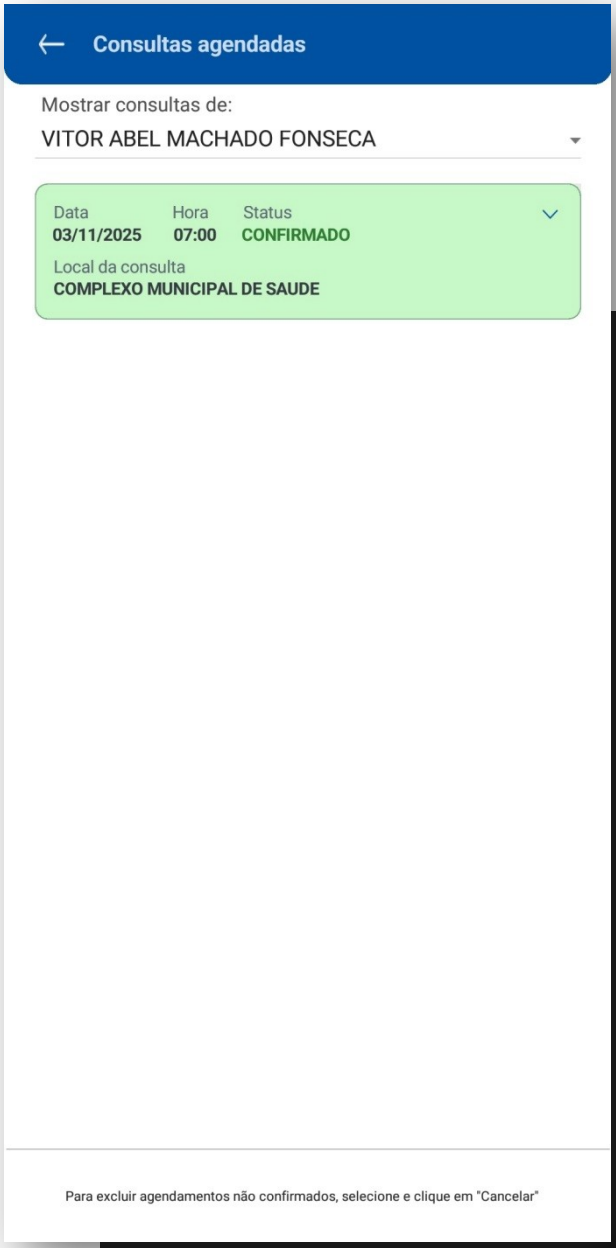
- Cria o formulário principal

  (TFrmPrincipal) se ainda não

  existir.Exibe a tela principal com

  FrmPrincipal.Show.

- Finalização do login (thread assíncrona):
- Em thread separada, chama:
- TrocaForm(FrmPrincipal) → muda visualmente da tela de login para a principal.
- FrmLogin.Release → libera a tela de login da memória.



Listar Consultas - Visão geral

1. **O app faz uma requisição GET para a rota pra trazer o cidadão:**
   GET -

[redacted]

http://[Conexao.Url]/cartao-Cidadao/[Conexao.Banco]/cidadao/[Usuario.CNS]/agendados

TdmConsulta é um **DataModule Delphi** que centraliza toda a **comunicação REST com o servidor** relacionada a **consultas médicas** (agendamentos, histórico, datas e horários disponíveis etc).
Ele usa o **RESTRequest4D** e **FireDAC** para buscar, confirmar, desmarcar e solicitar consultas, além de atualizar listas em memória no app.

Componentes principais

- **RESTClient / RESTRequest / RESTResponse** → componentes REST do Delphi para chamadas HTTP.
- **Eventos e propriedades:**
    - FOnConfirmarAgendamento: evento disparado após confirmação de presença.
    - AgendamentoSucesso: flag que indica se a solicitação de agendamento foi bem-sucedida.

Funções de pesquisa (GET)

Essas funções consultam a API e preenchem TFDMemTable e listas locais (ListaHistoricoConsulta, etc).

1. **PesquisarSolConsulta**

- Endpoint: /cidadao/{codigo}/agenda
- Lista todas as **consultas agendadas** do cidadão.
- Popula ListaHistoricoConsulta com data, hora, status, profissional, especialidade, local e observação.

2. **PesquisarConsultaPorCodigo**

- Endpoint: /cidadao/{codigo}/agenda/{codAgendamento}
- Retorna uma **consulta específica** pelo código.
- Preenche um registro único em ListaHistoricoConsulta.

3. **PesquisarSolConsultaAgendada**

- Endpoint: /cidadao/{codigo}/agendados
- Retorna consultas **já agendadas** (confirmadas).
- Preenche ListaHistoricoConsultaAgendada.

4. **PesquisarHistoricoConsulta**

- Endpoint: /cidadao/{codigo}/agenda/historico
- Retorna **histórico de consultas anteriores**.
- Popula ListaHistoricoConsulta com dados de consultas passadas.

5. **PesquisarDatasLivres**

- Endpoint: /agenda/{CNES}/{CBO}/{mês}/{ano}

- Retorna as **datas disponíveis** para agendamento.
- Popula ListaDatasLivres com as datas do mês/ano selecionados.

6. **PesquisarHorariosLivres**

- Endpoint: /agenda/{CNES}/{CBO}/{data}
- Retorna os **horários disponíveis** para a data escolhida.
- Popula ListaHorariosLivres.

7. **PesquisarProximaDataHorariosLivres**

- Endpoint: /agenda/{CNES}/{CBO}/horario/{data}/proximo
- Retorna a **próxima data com horários livres**.
- Popula ListaProximaDataHorariosLivres com data e horas.

Funções de escrita (POST)

Essas funções enviam dados JSON para o servidor (agendamento, confirmação, cancelamento).

SolicitarAgendamento

- Endpoint: /agenda/
- Envia JSON com dados do cidadão, unidade, especialidade, data e hora.
- Executa o POST.
- Analisa RESTResponse.StatusCode e mostra mensagem adequada:
    - **200:** sucesso → AgendamentoSucesso := True
    - **400–500:** exibe erro correspondente.

ConfirmarAgendamento

- Endpoint: /cidadao/{cidadao}/Confirmar/{codAgendamento}
- Envia JSON com cidadao e codage.
- Usa ExecuteAsync e chama o **evento** RESTRequestAfterExecute após resposta.
- Dispara FOnConfirmarAgendamento(Self, Sucesso) para atualizar interface.

DesmarcarSolicitacaoAgendamento

- Endpoint: /agenda/desmarcar
- Envia JSON com dados da consulta e motivo (obs).
- Mostra mensagem conforme código de retorno:
    - **200:** sucesso

- **400/401/404/500:** erro específico.

Eventos e controle

RESTRequestAfterExecute

- Executado após o POST de confirmação.
- Verifica RESTResponse.StatusCode = 200.
- Mostra mensagem de sucesso/erro.
- Dispara o evento OnConfirmarAgendamento (callback para telas como uFrmListaConsultas).

Fluxo geral do módulo

App solicita dados:
Telas como uFrmListaConsultas chamam PesquisarSolConsulta, PesquisarHistoricoConsulta, etc.

- **Usuário seleciona e agenda:**
  - SolicitarAgendamento envia a solicitação à API.
  - Em caso de sucesso, AgendamentoSucesso é marcado como True.
- **Usuário confirma presença:**
  - ConfirmarAgendamento envia o POST.
  - RESTRequestAfterExecute mostra resultado e dispara o evento de retorno.
- **Usuário desmarca consulta:**
  - DesmarcarSolicitacaoAgendamento envia a requisição de cancelamento.
- **Listas locais (em memória)** são sempre limpas e recarregadas conforme o endpoint.

Resumo funcional

Tipo	Método	Ação	Resultado
GET	PesquisarSolConsulta	Lista consultas atuais	ListaHistoricoConsulta
GET	PesquisarConsultaPorCodigo	Detalhe da consulta	ListaHistoricoConsulta
GET	PesquisarSolConsultaAgendada	Consultas confirmadas	ListaHistoricoConsultaAgendada
GET	PesquisarHistoricoConsulta	Histórico completo	ListaHistoricoConsulta
GET	PesquisarDatasLivres	Datas disponíveis	ListaDatasLivres
GET	PesquisarHorariosLivres	Horários disponíveis	ListaHorariosLivres
GET	PesquisarProximaDataHorariosLivres	Próxima data com horários	ListaProximaDataHorariosLivres
POST	SolicitarAgendamento	Envia solicitação	AgendamentoSucesso
POST	ConfirmarAgendamento	Confirma presença	Evento OnConfirmarAgendamento
POST	DesmarcarSolicitacaoAgendamento	Cancela consulta	Mensagem e atualização